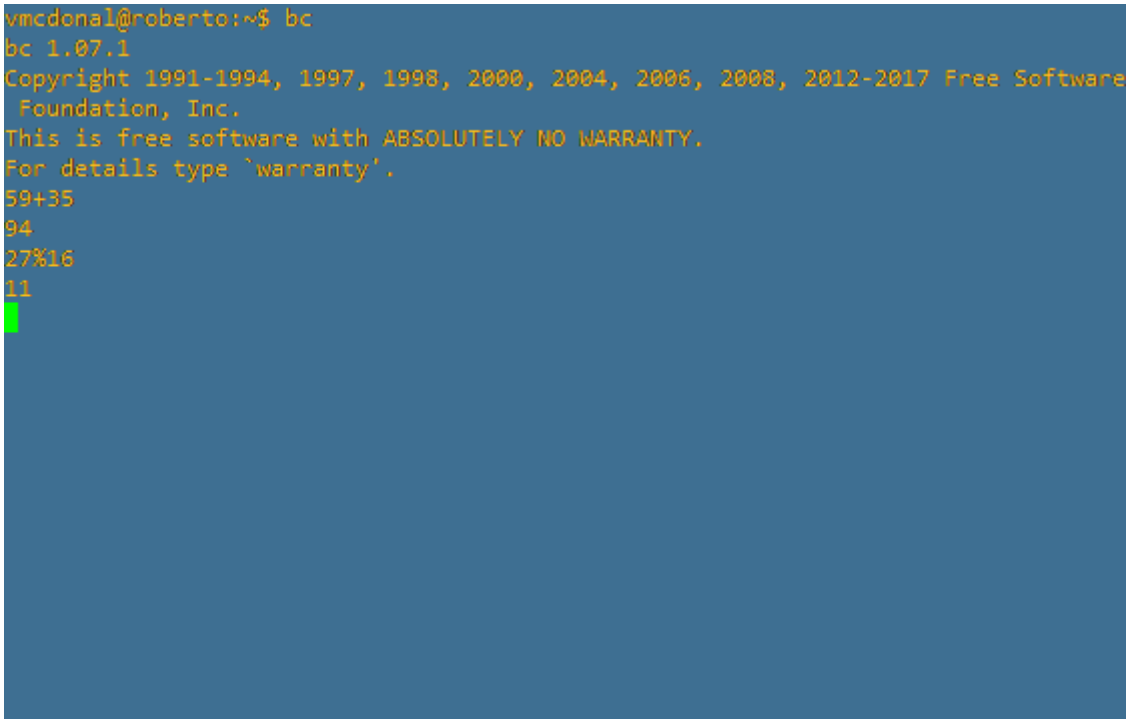
ID521 Digital Numeracy – Level 5 – 15 Credits

Assignment: Calculator

Software requirements:

- Your task is to write a simple command-line calculator which needs to successfully perform calculations using the mathematical methods covered in class. A command-line calculator is one where you present only a text interface, and the user enters their equations much like they would at a command shell or a DOS prompt.
    - For maximum marks you should implement them all
- You can work with others and use AI to understand the problems
- All submitted code **MUST** be your own work



Total possible marks: 50 (40% of your final mark for ID521)
Due Date: No later than 5pm Friday 26 June 2026

Task

You are to implement methods to perform calculations from *at least three* of the topics listed within this document. These topics are:

- Binary Numbers
- Geometry and Vectors
- Matrices
- Number Theory (bar codes and random number generation)
- Cryptography (Caesar and Affine cyphers)

Standard functions should use their standard symbology, e.g. + - * / !. In addition to these, as well as squares and square roots, you will implement **any three topics** from the list near the end of the document. The calculator is to use the lexicon as listed within this document.

The most basic version of your calculator will be a blank screen as above. How you proceed from that point is up to you, but the user needs to be able to enter the instructions and your calculator will either display the result, or an error message.

You may reuse any appropriate code from your Studio 1 game project. Please note it as such.

You've had a class on Unit Testing, and there are marks allocated for evidence of testing, as well as the techniques used.

Task Requirements

You will need to handle user input and correctly parse it, i.e. read in the user's instructions and breaking them down to the useful parts.

How you display your output is up to you but be consistent within your particular package.

Make sure to prevent undefined operations such as division by zero, where appropriate.

Implement BEDMAS where necessary, although it will be built into C# for most cases.

Within your source code, each method should have a comment above, explaining what the method does, what difficulties you faced, how you solved them and why you solved them the way you did.

Acknowledge and document any use of AI. Include your prompts and its responses in a document that is submitted alongside your code.

Use only standard libraries, not any libraries that you've installed separately.

Topics:

Implement **at least three** of the following topics:

Binary

- Binary addition and subtraction of signed and unsigned numbers. Display the result of these operations.
- Conversion between hexadecimal, binary, and decimal formats.
- Convert decimal numbers to the Binary Coded Decimal (BCD) format and display the result.
- Add BCD numbers and display the working and result.

Geometry and Vectors

- Allow the user to define coordinates sufficient to create a line.
- For a pair of coordinates, write methods to find the length of a line, the midpoint of the line, and the gradient of the line. Make sure to display the result of these operations.
- Allow the user to convert from radians to degrees, and degrees to radians, and display the result.
- Allow the user to define vectors and then write a method to convert a pair of coordinates to a vector, and store that vector. Display the result.
- Allow the user to get the length of the vector, perform arithmetic with another (i.e. add it to or subtract it from) vector, and display the vector that results from this equation. Also allow the user to perform scalar multiplication of a vector and display the result.
- Calculate the dot product of a vector and display the result.

Matrices

- Allow the user to define 2x2 matrices.
- Write functions to allow the user to add these matrices together, and display the result
- calculate the dot product of two matrices, and display the result
- perform scalar multiplication, and display the result
- calculate the determinant and the inverse of a matrix, and display the result

Number Theory

- Calculate the primality of a number given by the user.
- Calculate the check digits for UPC and EAN-13 bar codes, and ISBN. The user will provide you with the numbers for the bar code, but your program should figure out which it is, and perform the correct calculation method.
- Implement a Random Number Generator (RNG) using the Linear Congruential Method $X_{i+1} = (a X_i + c) \bmod m$.

Cryptography

- Implement encryption and decryption methods for the Caesar cypher.
- Implement encryption and decryption methods for the Affine cypher.
- Devise and implement a brute-force decryption algorithm for the Affine cypher. You will need to ask the user for an encrypted string and then print out both the encrypted and decrypted strings, the keys, and the total time it took to run the method. You do not need to programmatically determine that your decryption is successful, simply print the text out during each iteration.

Lexicon

Geometry

Line a (x1, y1, x2, y2)	create a line named a
lengthLine a	calculate the length of a line and display the result
midpointLine a	calculate the midpoint of a line and display the result
gradientLine a	calculate the gradient of a line and display the result
rad2Deg a	convert a from radians to degrees
deg2Rad a	convert a from degrees to radians

Vectors

vec a (x y)	create a vector named a, with the components x y
addVec a b	add together existing vectors a and b, and display the result
subVec a b	subtract existing vector a from b, and display the result (hint: this is the same as adding a negative number)
dotVec a b	calculate the dot product of a pair of vectors a and b. Display the result and the meaning of the result
scalVec s a	apply the scalar s to the vector a, and display the result

Matrices

mat a (a b c d)	create a 2x2 matrix named a, with the components a b c d
addMat a b	add matrices a and b, and display the result
dotMat a b	calculate the dot product for matrices a and b, and display the result

scalMat s a	apply the scalar s to the matrix a, and display the result
detMat a	calculate and display the determinant of matrix a
invMat a	calculate and display the inverse of the matrix a

Number Theory

numPrime a	determine if a number (less than 10000) is prime or not
numCheckDigit	given a numerical string, determine which code it is (UPC, EAN-13, or ISBN), calculate the check digit, and display the string
numRand a X c m	generate a random number using the Linear Congruential Method. Display the equation and variables at each step, in the format $X_{i+1} = (a X + c) \bmod m$ filling in the variables from the arguments as appropriate. (Subscripted characters can be display as normal text.)

Cryptography

Note: ignore spaces and punctuation when encrypting or decrypting

caesarEn “Encrypt”	Encode the provided string using the Caesar cypher: (n + 3) mod 26
caesarDe “Eluwkgdb sduwb fkhhvhfdnh mhoobehdq errp”	Decode the provided string using the Caesar cypher: (n – 3) mod 26
affineEn a b “Encrypt”	Encrypt the provided string using the Affine cypher: (aP + b) mod 26
affineDe a b “Csvpyfz”	Decrypt the provided string using the Affine cypher: (aP - b) mod 26
bruteAffine a b “Csvpyfz”	Decrypt the provided Affine-encrypted string using a brute force attack

Requirement	A 100-80%	B 79-65%	C 64-50%	Not Achieved <50%
Comments on Solution Choice (Learning Outcome 1) 15 marks Weighting: 30%	Comments justify choice of method for solving all problems, and provides analysis on method (e.g. trade-offs between different solutions) If used, AI prompts and responses supplied. Comments critique the AI's responses.	Comments justify choice of method for solving all problems. If used, AI prompts and responses supplied, and analysed.	Comments justify choice of algorithm used for solving calculation 3 problems. If used, AI prompts and responses supplied.	Sparse or missing comments. No or poor justification of choice of solution
Compile-time Errors (Learning Outcome 2) 5 marks Weighting: 10%	No compile-time errors of any kind. Code is clean, modular, and easy to maintain.	Compile-time errors limited to warnings or text-only, with no impact on execution	Minor compile-time errors that do not significantly impact execution of calculations or correctness of results	Errors in code or logic that prevent compilation or execution of functions
Testing and Methodology (Learning Outcome 1) 10 marks Weighting: 20%	Comprehensive Unit Testing covering valid, invalid, and edge cases. Comments explain why each test was chosen.	Testing covers most valid and invalid inputs. Tests verify full functionality. Comments should include reasoning for chosen tests	Testing verifies basic functionality with a range of valid inputs. Some invalid inputs tested. Comments are present but brief.	Little or no testing implemented. Missing input data or testing results. Testing is incomplete or absent.
Correctness of Results (Learning Outcome 2) 10 marks Weighting: 20%	Calculator returns expected results for valid inputs, also supplies help for instruction syntax when asked or an input error occurs (e.g. "Did you mean sqrt(9)")	Calculator returns expected results for valid inputs. Supplies help for instruction syntax when asked	Working calculator returns expected results for valid inputs	Calculator does not return expected results, or results are only partially correct
Correctness of Code (Learning Outcome 1) 10 marks Weighting: 20%	All topics are implemented correctly and fully. Code is modular and reusable.	Most topics are implemented correctly and fully.	At least 3 topics are correctly and fully implemented functions	Fails to implement a minimum of 3 topics fully or correctly